

LEVEL 0

Chapter 1: Prerequisites & Networking

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 1: Prerequisites & Networking Fundamentals

System Design Master Roadmap — Level 0 · [Index](#) · Next → [Chapter 2: System Design Fundamentals](#)

How to use this chapter

You already run production systems on AWS/EKS/Kafka, so you have *used* every protocol in this chapter without necessarily having the vocabulary to *explain* it under interview pressure. That's the gap this chapter closes. Skim fast where you're confident; slow down wherever you can't yet explain something out loud in 60 seconds without notes — that's the real bar, because in an interview you'll be explaining, not recalling.

Every topic below follows the same shape: **what it is → analogy → how it actually works → when it shows up in an interview → the one question that catches people out.**

1.1 The Internet in Five Layers (just enough networking)

You don't need a CCNA. You need enough to reason about *where latency comes from* and *why a request can fail*. Here's the whole stack you need, top to bottom:

Layer	What lives here	Why you care in an interview
Application	HTTP, gRPC, WebSocket, DNS	This is where your API design decisions live
Transport	TCP, UDP	Determines reliability vs. speed trade-offs
Network	IP	Routing — mostly invisible, but explains latency across regions
Data Link / Physical	Ethernet, Wi-Fi, fiber	Almost never discussed in HLD interviews — skip

IP (Internet Protocol) and Ports. Every machine reachable on a network has an IP address (like a street address — `142.250.183.14`). A single machine can run many services at once (a web server, a database, a cache), so **ports** (0-65535) act like apartment

numbers within that address — `142.250.183.14:443` is "this building, apartment 443" (HTTPS). Well-known ports worth memorizing: 80 (HTTP), 443 (HTTPS), 22 (SSH), 3306 (MySQL), 5432 (PostgreSQL), 6379 (Redis), 9092 (Kafka), 27017 (MongoDB).

Interview relevance: Low — this is background knowledge that makes everything else make sense. It rarely gets asked directly, but "which port does X run on" signals whether you've actually operated these systems, which you have.

TCP vs UDP — the decision that shapes your whole design

This is one of the few "prerequisite" topics that genuinely gets asked, because the choice cascades into your entire real-time architecture.

TCP (Transmission Control Protocol) — a phone call. Before any data flows, both sides do a **3-way handshake** (SYN → SYN-ACK → ACK) to agree they're both listening. Every packet is acknowledged; lost packets are retransmitted; packets are reassembled in order on the receiving end. This reliability costs latency (handshake round-trip + retransmission delays) and overhead (headers, ACKs).

UDP (User Datagram Protocol) — dropping a postcard in the mail. No handshake, no guarantee of delivery, no guaranteed order. It's fast and cheap precisely because it doesn't do any of TCP's bookkeeping.

	TCP	UDP
Connection	Handshake required	Connectionless
Reliability	Guaranteed delivery + order	Best-effort, no guarantees
Speed	Slower (overhead)	Faster (minimal overhead)
Use when	Data must arrive intact: HTTP APIs, database connections, file transfer, chat messages	Losing a few packets is fine and speed matters more: video/voice calls, live location pings, multiplayer games, DNS queries

Interview question: "Why would you use UDP for live location tracking in a ride-hailing app instead of TCP?"

Ideal senior answer: "Location pings arrive every 2–4 seconds. If one packet is lost, the next one two seconds later makes it irrelevant — retransmitting a stale coordinate wastes bandwidth and adds latency for no benefit. TCP's ordering and retry guarantees are actively harmful here: I'd rather drop a stale point and get the freshest one faster. I'd use UDP-based transport (or WebSocket-over-TCP if I need the connection semantics but can tolerate the overhead, which is the common practical compromise) with an application-level 'use latest timestamp wins' rule."

Common mistake: Saying "always use TCP for reliability" without recognizing that reliability isn't free — for time-sensitive, replaceable data, reliability is the wrong optimization target.

DNS — how a name becomes an address

Simple explanation: DNS is the internet's phonebook. You type `api.swiggy.com`; DNS resolves it to an IP address like `13.234.10.22` so your machine knows where to send packets.

How it actually works (the resolution chain):

1. Browser checks its own cache, then OS cache.
2. If not cached, a **recursive resolver** (usually your ISP's or `8.8.8.8`) takes over.
3. It asks a **root nameserver** → which points to the **TLD nameserver** (for `.com`) → which points to the **authoritative nameserver** for `swiggy.com` → which returns the actual IP.
4. The result is cached at every layer according to its **TTL** (time-to-live), so this whole chain usually only runs once per TTL window, not per request.

Why it matters in system design interviews: DNS is your first, cheapest load-balancing and traffic-routing tool. **Round-robin DNS** hands out different IPs to spread load. **GeoDNS** routes users to the nearest regional deployment — this is how multi-region architectures direct European users to `eu-west-1` and Indian users to `ap-south-1` before a single request hits your load balancer. Low TTLs (60s) let you fail over fast during an incident; high TTLs (24h) reduce DNS query load but slow down failover.

Common mistake: Treating DNS as instant and free. DNS lookup is a real network round trip (tens to hundreds of ms on cold cache) and belongs in your latency budget when you're estimating end-to-end response time for a first-time client.

1.2 HTTP, HTTPS, and TLS

HTTP (HyperText Transfer Protocol) is a request/response protocol built on top of TCP. A client sends a request (method + path + headers + optional body); a server sends back a response (status code + headers + body). It's **stateless** by design — the server doesn't remember your previous request unless you explicitly add state (cookies, sessions, tokens). This statelessness is *the* reason horizontal scaling of web servers is easy: any server can handle any request.

Know these status code families cold, because interviewers use them as a shorthand:

Range	Meaning	Examples you should know
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified (cache validation)
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

HTTPS = HTTP + TLS. It's the same protocol, wrapped in an encrypted tunnel.

TLS (Transport Layer Security) solves three problems at once: **encryption** (nobody snooping on the wire can read the data), **integrity** (nobody can tamper with it undetected), and **authentication** (you can verify you're actually talking to `api.razorpay.com` and not an impostor). It does this via the **TLS handshake**: the client and server exchange certificates, agree on a shared symmetric key using asymmetric cryptography, and then switch to fast symmetric encryption for the actual data (asymmetric crypto is too slow to use for every byte).

Interview question: "Where does TLS get terminated in your architecture, and why does it matter?"

Ideal senior answer: "Usually at the load balancer or CDN edge — that's 'TLS termination.' It matters because decrypting is CPU-expensive, and doing it once at the edge instead of at every backend instance saves compute and lets me centralize certificate management. The trade-off is that traffic between the load balancer and backend is now unencrypted unless I explicitly re-encrypt it — which I'd want for anything handling payment data or PII, even inside a VPC, to satisfy defense-in-depth and compliance requirements like PCI-DSS."

Common mistake: Assuming "inside the VPC" automatically means "secure." Regulated systems (fintech, healthcare) usually require encryption in transit even between internal services.

1.3 How Services Talk to Each Other: REST, gRPC, WebSockets

This is genuinely an interview topic (Level 8 goes much deeper), but you need the vocabulary now because it shows up in every architecture diagram from here on.

REST (Representational State Transfer) — an architectural style over HTTP where you model your system as **resources** (`/users/123/orders`) manipulated with standard verbs (GET, POST, PUT, PATCH, DELETE). It's human-readable, cacheable (thanks to HTTP semantics), and universally supported — which is exactly why it's the default choice for public/partner-facing APIs.

gRPC — a binary RPC framework built on HTTP/2, using Protocol Buffers (protobuf) for serialization. You define a service contract in a `.proto` file, and it generates strongly-typed client/server code in many languages. It's significantly faster and smaller on the wire than JSON-over-REST, and supports **streaming** (client, server, or bidirectional) natively. The cost: it's not human-readable in transit, harder to debug with `curl`, and less friendly for public-facing browser clients (though `grpc-web` exists).

WebSockets — a protocol that starts as an HTTP request (`Upgrade: websocket`) and then upgrades the TCP connection into a **persistent, full-duplex** channel. Once upgraded, either side can push messages at any time without the request/response ceremony. This is what powers chat apps, live dashboards, and multiplayer features.

	REST	gRPC	WebSocket
Best for	Public APIs, CRUD, browser clients	Internal service-to-service, low-latency, streaming	Real-time bidirectional push (chat, live tracking)
Payload	JSON (text, larger)	Protobuf (binary, smaller)	Any (often JSON or binary frames)
Connection	New/reused per request	Persistent (HTTP/2 multiplexed)	Persistent
Browser support	Native	Needs <code>grpc-web</code> /proxy	Native
Human debuggable	Yes	No (needs tooling)	Partially

Interview question: "You have 40 internal microservices talking to each other synchronously. Would you use REST or gRPC?"

Ideal senior answer: "For internal service-to-service calls I'd lean gRPC — lower latency from binary serialization and HTTP/2 multiplexing, and the generated client stubs from a shared `.proto` schema prevent the kind of contract drift you get with hand-maintained REST clients across 40 services. I'd keep REST at the edge — the public/partner-facing API gateway — because it's more universally consumable and easier to version and document for external consumers. So the real answer is both, at different boundaries, not one framework for the whole system."

Follow-up you should expect: "What if a service needs to push events to 10,000 connected mobile clients?" → Neither REST nor gRPC alone; you'd want a **pub/sub layer + WebSocket/SSE gateway** — covered in Chapter 21 (Real-Time Systems).

(REST vs GraphQL vs gRPC gets a much deeper treatment, including good/bad API examples, versioning, pagination, and idempotency, in [Chapter 09: API Design](#).)

1.4 Traffic-Control Building Blocks (quick preview)

You'll get a full chapter on each of these (Chapter 4), but you need the one-line mental model now because they appear in every architecture diagram from Chapter 2 onward:

Component	One-line mental model
Reverse Proxy	Sits in front of your servers, forwards client requests to them, and returns the response — clients never talk to your servers directly.
Load Balancer	A reverse proxy whose specific job is spreading traffic across <i>many</i> identical backend instances so no single one is overwhelmed.
API Gateway	A smarter reverse proxy at the edge that also does auth, rate limiting, routing to different services, and request/response transformation — the single front door to your microservices.
CDN (Content Delivery Network)	A globally distributed network of caching servers ("edge locations") that serve static (and increasingly dynamic) content from a location physically close to the user, so requests don't have to travel to your origin server every time.

1.5 Identity: Cookies, Sessions, JWT, OAuth, AuthN vs AuthZ

The core problem these all solve: HTTP is stateless. Without help, your server has no idea request #2 came from the same user as request #1. Every mechanism below is a different answer to "how do we remember who's talking to us."

Cookies — a small piece of data the server tells the browser to store (`Set-Cookie` header) and send back automatically on every subsequent request to that domain. Cookies are the *transport mechanism*; what you put inside them is up to you.

Sessions — the classic pattern: the server creates a session record (in memory, Redis, or a DB) keyed by a random **session ID**, and sends only that ID to the client as a cookie. The server looks up the session ID on every request to retrieve the actual user state. This is **stateful** on the server side — which is exactly why it complicates horizontal scaling: any server handling a request needs access to that session store, so you either use **sticky sessions** (route the same user to the same server — fragile) or a **shared session store** like Redis (better, but now Redis is a dependency every request touches).

JWT (JSON Web Token) — flips the model: instead of storing state on the server and handing the client a reference (session ID), you encode the state *itself* into a signed token and hand that to the client. A JWT has three parts (`header.payload.signature`), base64-encoded and dot-separated. The server verifies the signature on each request — no database lookup needed. This is what makes JWTs attractive for **stateless, horizontally-scaled** systems and microservices, where you don't want every service hitting a shared session store just to know who's calling.

	Session (server-side state)	JWT (client-side state)
Server lookup per request	Yes (session store)	No (signature verification only)
Revocation before expiry	Easy (delete session record)	Hard (needs a blocklist, defeats the purpose)
Payload size on wire	Small (just an ID)	Larger (whole payload every request)
Best for	Traditional web apps, when instant revocation matters	Stateless APIs, microservices, mobile clients

Interview question: "Your payments team says they need to instantly revoke a compromised user's access. You're using JWTs everywhere. What do you do?"

Ideal senior answer: "This is the classic JWT trade-off — stateless tokens are fast to verify but hard to revoke early, since the token is valid until it expires no matter what the server thinks. Practically: keep access-token TTLs short — 5 to 15 minutes — so a compromised token expires quickly on its own, back it with a longer-lived refresh token that *is* checked against a server-side store (Redis) on refresh, and maintain a short-lived revocation blocklist in Redis for the rare 'kill this token right now' case, checked only on high-privilege actions rather than every request to avoid re-introducing a lookup on the hot path everywhere."

OAuth 2.0 — not an authentication protocol per se; it's an **authorization delegation** protocol. It's how you let a third-party app access your data on another service *without* giving it your password — think "Sign in with Google" or letting a food-delivery app read your Google contacts. The core flow (Authorization Code flow): your app redirects the user to Google → user approves → Google redirects back with a short-lived **authorization code** → your backend exchanges that code (plus a client secret) for an **access token** → your app uses that access token to call Google's APIs on the user's behalf.

OIDC (OpenID Connect) is a thin identity layer *on top of* OAuth2 that adds the actual "who is this user" piece (the ID token) — this is the part people usually mean when they say "login with Google," even though they say "OAuth."

Authentication vs. Authorization — the distinction interviewers check for:

- **Authentication (AuthN)** — "Who are you?" Verifying identity (login, password check, token validation).
- **Authorization (AuthZ)** — "What are you allowed to do?" Checking permissions after identity is established (can this user delete this order?).

People conflate these constantly. A 401 Unauthorized response actually means "I don't know who you are" (authentication failure); a 403 Forbidden means "I know who you are, and you're not allowed to do this" (authorization failure). Getting this right in an interview is a small but real signal of precision.

(RBAC, ABAC, API keys, password hashing, and the full security surface get their own deep dive in [Chapter 10: Security](#).)

1.6 Chapter 1 Interview Drill

Before moving on, you should be able to answer each of these out loud, in under 90 seconds, without notes:

1. Walk me through what happens between typing a URL and seeing a page — every hop.
2. Why is HTTP called "stateless," and what problem does that create?
3. When would you pick UDP over TCP, with a concrete example?
4. What's the practical difference between a session-based auth system and a JWT-based one, and when would you pick each?
5. What's the difference between authentication and authorization — give an HTTP status code for each failure mode.
6. Why might an internal microservice call use gRPC while the public API uses REST?

If any of these make you pause, re-read that section — everything after this chapter assumes this vocabulary is automatic.

Common Mistakes at This Level

Mistake	Why it hurts you	Fix
Saying "REST is always better" or "gRPC is always better"	Signals you haven't used either in a real system with real trade-offs	Always anchor the choice to the specific communication pattern (public vs internal, streaming vs request/response)
Confusing authentication and authorization	Interviewers use this as a quick filter	Practice the one-liner: AuthN = who you are, AuthZ = what you can do
Treating DNS/TLS handshake latency as zero	Leads to wildly optimistic latency estimates later	Budget 20-150ms for DNS + TLS handshake on cold connections in capacity/latency discussions
Not knowing why JWTs are hard to revoke	A very common senior-level follow-up trap	Know the short-TTL + refresh-token + blocklist answer above

Next → [Chapter 2: System Design Fundamentals](#) — HLD vs LLD, the CAP theorem, scalability, and the vocabulary every other chapter builds on.